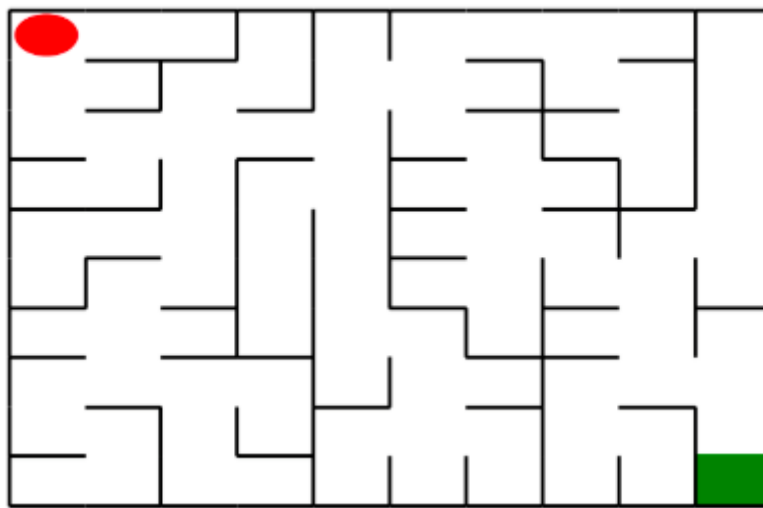


机器人走迷宫-实训报告

2312966 林晖鹏

一、问题重述

问题背景：



如上图所示，左上角的**红色椭圆**既是起点也是机器人的初始位置，右下角的**绿色方块**是出口。游戏规则为：从起点开始，通过错综复杂的迷宫，到达目标点(出口)。

在任一位置可执行动作包括：向上走'u'、向右走'r'、向下走'd'、向左走'l'。

- 执行不同的动作后，根据不同的情况会获得不同的奖励，具体而言，有以下几种情况。
 - 撞墙
 - 走到出口
 - 其余情况

你需要实现基于**基础搜索算法**和**Deep QLearning**算法的机器人，使机器人自动走到迷宫的出口。

实验要求：

- 使用 Python 语言。
- 使用基础搜索算法完成机器人走迷宫。
- 使用 Deep QLearning 算法完成机器人走迷宫。
- 算法部分需要自己实现，不能使用现成的包、工具或者接口。

二、设计思想

1. 基础搜索算法

这里提供了两个实现思路的选择：DFS深度优先搜索算法和A*最佳优先搜索。我们这里实现DFS算法：

完整代码详情看下一节代码内容部分。这里只对核心代码解释：

- `isvisit_m`：这个是一个二维矩阵，用来记录哪些位置是到达过的，和普通的DFS思路一致，到达过的位置我们不再遍历这个节点。
- `path`：记录路径
- `start`：记录目前节点的位置

1. 先判断到达终点没

代码块

```
1  if start == maze.destination:
2      return True, path, isvisit_m
```

- 2. 获取这个节点能走的方向，然后开始遍历这个节点的动作分支。先获取新的位置，如果这个位置没有被遍历过才走，然后递归遍历这个子节点。直到`p==1`，也就是到达终点的时候，返回。

代码块

```
1  can_move = maze.can_move_actions(start)
2
3  for i in can_move:
4      new_loc = tuple(start[j] + move_map[i][j] for j in range(2))
5
6      if not isvisit_m[new_loc]:
7          isvisit_m[new_loc] = 1
8
9          new_path = path.copy()
10         new_path.append(i)
11
12         p, end_path, isvisit_m = DFS(maze, new_path, new_loc, isvisit_m)
13         if p == 1:
14             return p, end_path, isvisit_m
```

2. Deep QLearning算法

由于当地图相当大的时候，Q表的大小成 $O(n^2)$ 的级别增长，对于学习更新以及内存都是一个很大的负担，所以DQN算法被提出，DQN使用深度神经网络来逼近Q函数，输入状态（本实验是输入位置），输出每个动作的Q值。

1. Q学习基础：

- DQN基于Q学习，Q学习的目标是学习一个Q函数，表示在状态 (s) 下采取动作 (a) 的预期未来奖励。
- Q值更新公式：
$$[Q(s, a) \rightarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]]$$
 其中，(r) 是即时奖励，(γ) 是折扣因子，(α) 是学习率，(s') 是下一状态。

2. 深度神经网络逼近：

- 传统Q学习使用表格存储Q值，适合状态和动作空间较小的环境。但在复杂环境中（如游戏），状态空间巨大，表格无法应对。
- DQN使用深度神经网络（通常为卷积神经网络）来逼近Q函数，输入状态（如图像），输出每个动作的Q值。

3. 经验回放 (Experience Replay)：

- DQN将智能体的经历（状态、动作、奖励、下一状态）存储在经验池中。
- 训练时，从经验池中随机抽取小批量数据，打破时间相关性，提高样本利用率和训练稳定性。

4. 目标网络 (Target Network)：

- DQN使用两个神经网络：当前网络（用于选择动作）和目标网络（用于计算目标Q值）。
- 目标网络参数定期从当前网络复制，减少训练过程中的Q值震荡，提升稳定性。

5. 损失函数：

- DQN通过最小化预测Q值与目标Q值之间的均方误差来优化网络：
$$[L = \mathbb{E} [\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2]]$$
 其中，(θ) 是当前网络参数，(θ^-) 是目标网络参数。

下面，我们直接从代码开始分析算法。

2.1 DQN网络的设置

Pytorch中设计的DQN机器人的网络架构只有两层，过于简单，一方面难以学习复杂且庞大的网络，另一方面容易对某部分过拟合的同时，对地图其余部分欠拟合。所以这里我们换成三层的前馈神经网络架构。

代码块

```
1  # 定义DQN网络
2  class DQN(nn.Module):
```

```

3     def __init__(self, state_size, action_size, seed):
4         super(DQN, self).__init__()
5         torch.manual_seed(seed)
6         self.fc1 = nn.Linear(state_size, 128)
7         self.fc2 = nn.Linear(128, 128)
8         self.fc3 = nn.Linear(128, action_size)
9
10    def forward(self, x):
11        x = F.relu(self.fc1(x))
12        x = F.relu(self.fc2(x))
13        return self.fc3(x)

```

2.2 机器人参数初始化

实验过程中，做了很久，感觉实验的核心还是调整参数，所以这里参数的设置至关重要。

- `maze_size`：迷宫大小。
- `batch_size`：每次训练批次大小。这个如果太大，则难以进入训练，因为经验池中经验不够；而太小的话，又难以学习地图特征。
- `learning_rate`：学习率。太小的话需要让训练轮次增大。
- `epsilon0`：初始的随机游走概率。DQN中为了增加模型在训练过程中的探索能力，增加了随机游走的策略。
- `target_update`：目标网络更新的频率。
- `max_size`：经验池的大小
- `gamma`：折扣因子。

2.2.1 探索策略的调整

在实际实验的过程中，我们发现这个随机游走的策略在不同地图大小下有着不同的表现。当地图过大的时候，我们下降初始的随机游走概率，主要还是依靠学习。

2.2.2 预训练

我爱说实话，这里限制训练轮次就离谱，小轮次对于大地图根本就走不到终点。看了很多个学长学姐的报告，都是采用while循环直到走到终点的策略，让模型强行在小轮次下能学习到终点的Q值。

而这里我们就不那么离谱了，确实while的话，还不如用DFS呢，时间花销太久啦。我们这里提前预训练一次，其实本质就是增加训练的轮次。让正式训练的时候，Q网络得出的初始Q值能够更加好。

这里也用了分段的思想，当地图过大的时候，我们训练轮次也大一点。

代码块

```

1     def __init__(self, maze):
2         """

```

```

3         初始化 Robot 类
4         :param maze: 迷宫对象
5         """
6         super(Robot, self).__init__(maze)
7         self.maze = maze
8         self.maze_size = maze.maze_size
9         self.state_size = 2
10        self.action_size = 4
11        self.batch_size = 64
12        self.learning_rate = 1e-3
13        self.gamma = 0.95
14        self.epsilon0 = 0.99
15        self.epsilon = self.epsilon0
16        self.epsilon_min = 0.01
17        self.step = 0
18        self.target_update = 70 # 目标网络更新频率
19        self.device = torch.device("cuda:0" if torch.cuda.is_available() else
"cpu")
20
21        self.maze.set_reward(reward={
22            "hit_wall": -10.,
23            "destination": 50.,
24            "default": -0.5,
25        })
26
27        # 创建经验回放存储
28        max_size = min(self.maze_size ** 2 * 3, 10000)
29        self.memory = ReplayDataSet(max_size=max_size)
30
31
32        # 创建主网络和目标网络
33        seed = random.randint(1, 100)
34        self.eval_model = DQN(self.state_size, self.action_size,
seed).to(self.device)
35        self.target_model = DQN(self.state_size, self.action_size,
seed).to(self.device)
36        self.target_model.load_state_dict(self.eval_model.state_dict())
37        self.target_model.eval()
38
39        # 定义优化器
40        self.optimizer = optim.Adam(self.eval_model.parameters(),
lr=self.learning_rate)
41
42        # 动作映射
43        self.action_map = {0: "u", 1: "r", 2: "d", 3: "l"}
44        self.action_index_map = {"u": 0, "r": 1, "d": 2, "l": 3}
45

```

```

46         if self.maze_size == 5:
47             self.memory.build_full_view(maze=maze)
48             runner = Runner(self)
49             runner.run_training(10, int(self.maze_size ** 2 * 1.5))
50
51         # 当迷宫大小过大时, 调整epsilon
52         if self.maze_size >= 10:
53             self.target_update = 60
54             self.epsilon = 0.3
55             self.epsilon0 = 0.3
56             self.maze.set_reward(reward={
57                 "hit_wall": -10.,
58                 "destination": 50.,
59                 "default": -0.5,
60             })
61             self.memory.build_full_view(maze=maze)
62             runner = Runner(self)
63             runner.run_training(50, int(self.maze_size ** 2 * 1.5))

```

3. 选择动作函数

- a. 我们先获取能走的方向, 然后判断是不是空 (实际上完全不可能为空, 总不能回头路都没了吧?)。
- b. 生成一个随机数, 如果小于随机概率, 就实行随机游走策略, 否则用行动网络生成Q值, 选取Q值最大的方向。
- c. 返回执行动作

代码块

```

1  def _choose_action(self, state, is_training=True):
2      valid_actions = self.maze.can_move_actions(state)
3      if not valid_actions:
4          valid_actions = self.valid_action
5
6      if is_training and random.random() < self.epsilon:
7          # 探索: 随机选择有效动作
8          return random.choice(valid_actions)
9      else:
10         # 利用: 选择Q值最大的有效动作
11         state_tensor = self._state_to_tensor(state)
12         self.eval_model.eval()
13         with torch.no_grad():
14             q_values = self.eval_model(state_tensor).cpu().data.numpy()
15         self.eval_model.train()
16

```

```

17         valid_indices = [self.action_index_map[a] for a in valid_actions]
18         max_q = float('-inf')
19         best_action = valid_actions[0]
20
21         for idx in valid_indices:
22             if q_values[idx] > max_q:
23                 max_q = q_values[idx]
24                 best_action = self.action_map[idx]
25
26         return best_action

```

4. 学习函数

- a. 判断经验池是否有足够经验
- b. 取出一个批次的经验
- c. 计算当前网络的Q值，选出最好的动作
- d. 用目标网络计算下一个状态的Q值，然后算出当前动作的更新Q值
- e. 计算损失然后更新目标网络

这里可能会由于来回走导致Q值叠加而发生梯度爆炸，所以我们这里还设置了梯度裁剪的策略。

代码块

```

1  def _learn(self, batch_size):
2      """优化模型"""
3      if len(self.memory) < batch_size:
4          return 0.0
5
6      states, actions, rewards, next_states, is_terminals =
self.memory.random_sample(batch_size)
7
8      # 转换为张量
9      states = torch.FloatTensor(states).to(self.device)
10     actions = torch.LongTensor(actions).to(self.device).reshape(-1, 1)
11     rewards = torch.FloatTensor(rewards).to(self.device).reshape(-1, 1)
12     next_states = torch.FloatTensor(next_states).to(self.device)
13     is_terminals = torch.FloatTensor(is_terminals).to(self.device).reshape(-1,
14     1)
15
16     # 计算目标Q值 (Double DQN)
17     self.eval_model.train()
18     self.target_model.eval()
19     with torch.no_grad():
20         next_action_indices = self.eval_model(next_states).max(1)
21         [1].unsqueeze(1)

```

```

20         q_targets_next = self.target_model(next_states).gather(1,
next_action_indices)
21         q_targets = rewards + self.gamma * q_targets_next * (1 - is_terminals)
22
23         # 计算当前Q值
24         q_expected = self.eval_model(states).gather(1, actions) # 形状:
[batch_size, 1]
25         # 计算损失
26         loss = F.mse_loss(q_expected, q_targets)
27         # 优化模型
28         self.optimizer.zero_grad()
29         loss.backward()
30         # 由于Q值可能会叠加导致爆炸, 这里进行梯度裁剪
31         for param in self.eval_model.parameters():
32             param.grad.data.clamp_(-1, 1)
33         self.optimizer.step()
34
35         return loss.item()

```

5. 训练函数

主要流程就是：

- a. 获取状态并选择、执行动作；
- b. 获取执行动作后的状态
- c. 判断是否终止
- d. 送入经验池
- e. 训练当前网络
 - 这里我们根据地图的大小来设置批次大小：`batch_size = min(32, (self.maze_size - 1) ** 2)`
- f. 周期性更新目标网络
- g. 探索率衰减
 - 这里我们不同于正常模型，我们采用线性衰减而不是指数衰减。

代码块

```

1  def train_update(self):
2      # 获取当前状态
3      state = self.sense_state()
4
5      # 选择动作
6      action = self._choose_action(state, is_training=True)

```



```

7
8     # 执行动作
9     reward = self.maze.move_robot(action)
10
11     # 获取下一状态
12     next_state = self.sense_state()
13
14     # 判断是否终止
15     is_terminal = 1 if next_state == self.maze.destination or next_state
    == state else 0
16
17     # 存储经验
18     action_idx = self.action_index_map[action]
19     self.memory.add(state, action_idx, reward, next_state, is_terminal)
20
21     # 优化模型
22     batch_size = min(32, (self.maze_size - 1) ** 2)
23     loss = self._learn(batch_size)
24
25     # 更新目标网络
26     self.step += 1
27     if self.step % self.target_update == 0:
28         self.target_model.load_state_dict(self.eval_model.state_dict())
29
30     # 衰减探索率, 这里我们选择线性衰减
31     delta = 0.000375
32     if self.maze_size == 3:
33         delta = 0.0015
34     elif self.maze_size == 5:
35         delta = 0.00075
36     self.epsilon = max(self.epsilon_min, self.epsilon - delta)
37
38     return action, reward

```

6. 测试函数

和训练流程差不多, 获取状态->选择动作->执行动作->返回动作和期望回报。

代码块

```

1  def test_update(self):
2      """
3      以测试状态选择动作
4      :return: action, reward 如: "u", -1
5      """
6      # 获取当前状态
7      state = self.sense_state()

```

```
8
9     # 选择动作
10    action = self._choose_action(state, is_training=False)
11
12    # 执行动作
13    reward = self.maze.move_robot(action)
14
15    return action, reward
```

三、代码内容

1. 基础搜索算法

代码块

```
1  import numpy as np
2
3  # 机器人移动方向
4  move_map = {
5      'u': (-1, 0), # up
6      'r': (0, +1), # right
7      'd': (+1, 0), # down
8      'l': (0, -1), # left
9  }
10 def DFS(maze, path, start, isvisit_m):
11     """
12     深度优先搜索算法
13     :param :
14         path之前的路径
15         start现在的位置
16         isvisit_m已到达的位置
17     :return :
18         bool是否到达终点
19         path路径
20         isvisit_m已到达位置
21     """
22     # 如果已经到达目标点
23     if start == maze.destination:
24         return True, path, isvisit_m
25
26     can_move = maze.can_move_actions(start)
27
28     # 如果没有可以走的了, 但是不可能
```

```

29     if len(can_move) == 0:
30         return false, path, isvisit_m
31
32     # 深搜开始
33     for i in can_move:
34         # print(can_move)
35         new_loc = tuple(start[j] + move_map[i][j] for j in range(2))
36         # 如果没有被走过
37         if not isvisit_m[new_loc]:
38             isvisit_m[new_loc] = 1
39
40             new_path = path.copy()
41             new_path.append(i)
42
43             p, end_path, isvisit_m = DFS(maze, new_path, new_loc, isvisit_m)
44             if p == 1:
45                 return p, end_path, isvisit_m
46
47     return False, path, isvisit_m
48 def my_search(maze):
49     """
50     任选深度优先搜索算法、最佳优先搜索 (A*)算法实现其中一种
51     :param maze: 迷宫对象
52     :return :到达目标点的路径 如: ["u","u","r",...]
53     """
54     path = []
55     # -----请实现你的算法代码-----
56     # 获取当前位置
57     start = maze.sense_robot()
58     # 迷宫大小
59     h, w, _ = maze.maze_data.shape
60     # 标记迷宫的各个位置是否被访问过
61     is_visit_m = np.zeros((h, w), dtype=np.int)
62     is_visit_m[0][0]=1
63
64     p, path, isvisit_m = DFS(maze, path, start, is_visit_m)
65     # -----
66     return path
67

```

2. Deep QLearn算法

代码块

```

1  from QRobot import QRobot
2  from ReplayDataSet import ReplayDataSet

```

```

3  import numpy as np
4  import random
5  import torch
6  import torch.nn as nn
7  import torch.nn.functional as F
8  import torch.optim as optim
9
10 # 定义DQN网络
11 class DQN(nn.Module):
12     def __init__(self, state_size, action_size, seed):
13         super(DQN, self).__init__()
14         torch.manual_seed(seed)
15         self.fc1 = nn.Linear(state_size, 128)
16         self.fc2 = nn.Linear(128, 128)
17         self.fc3 = nn.Linear(128, action_size)
18
19     def forward(self, x):
20         x = F.relu(self.fc1(x))
21         x = F.relu(self.fc2(x))
22         return self.fc3(x)
23
24 class Robot(QRobot):
25     valid_action = ['u', 'r', 'd', 'l']
26
27     def __init__(self, maze):
28         """
29         初始化 Robot 类
30         :param maze: 迷宫对象
31         """
32         super(Robot, self).__init__(maze)
33         self.maze = maze
34         self.maze_size = maze.maze_size
35         self.state_size = 2
36         self.action_size = 4
37         self.batch_size = 64
38         self.learning_rate = 1e-3
39         self.gamma = 0.95
40         self.epsilon0 = 0.99
41         self.epsilon = self.epsilon0
42         self.epsilon_min = 0.01
43         self.step = 0
44         self.target_update = 70 # 目标网络更新频率
45         self.device = torch.device("cuda:0" if torch.cuda.is_available() else
"cpu")
46
47         self.maze.set_reward(reward={
48             "hit_wall": -10.,

```

```

49         "destination": 50.,
50         "default": -0.5,
51     })
52
53     # 创建经验回放存储
54     max_size = min(self.maze_size ** 2 * 3, 10000)
55     self.memory = ReplayDataSet(max_size=max_size)
56
57
58     # 创建主网络和目标网络
59     seed = random.randint(1, 100)
60     self.eval_model = DQN(self.state_size, self.action_size,
61 seed).to(self.device)
62     self.target_model = DQN(self.state_size, self.action_size,
63 seed).to(self.device)
64     self.target_model.load_state_dict(self.eval_model.state_dict())
65     self.target_model.eval()
66
67
68     # 定义优化器
69     self.optimizer = optim.Adam(self.eval_model.parameters(),
70 lr=self.learning_rate)
71
72
73     # 动作映射
74     self.action_map = {0: "u", 1: "r", 2: "d", 3: "l"}
75     self.action_index_map = {"u": 0, "r": 1, "d": 2, "l": 3}
76
77
78     if self.maze_size == 5:
79         self.memory.build_full_view(maze=maze)
80         runner = Runner(self)
81         runner.run_training(10, int(self.maze_size ** 2 * 1.5))
82
83
84     # 当迷宫大小过大时, 调整epsilon
85     if self.maze_size >= 10:
86         self.target_update = 60
87         self.epsilon = 0.3
88         self.epsilon0 = 0.3
89         self.maze.set_reward(reward={
90             "hit_wall": -10.,
91             "destination": 50.,

```

四、实验结果

测试结果如下图，四个都通过了！

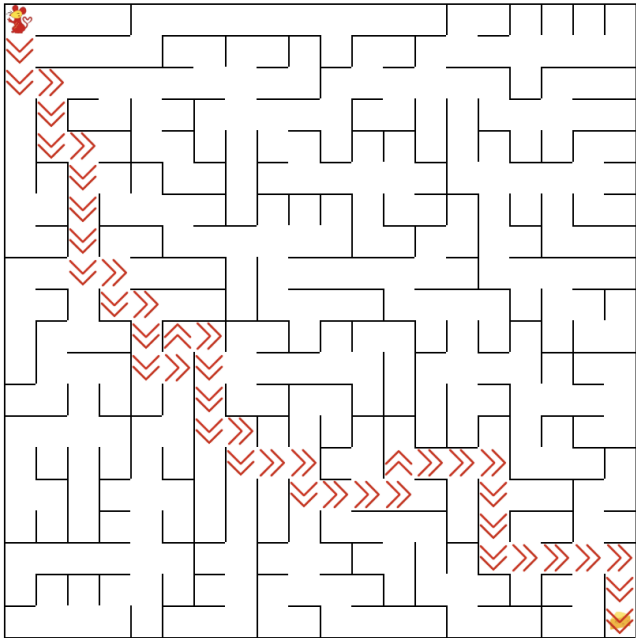
测试详情 展示迷宫

测试点	状态	时长	结果
测试基础搜索算法	✓	1s	恭喜, 完成了迷宫
测试强化学习算法(初级)	✓	1s	恭喜, 完成了迷宫
测试强化学习算法(中级)	✓	1s	恭喜, 完成了迷宫
测试强化学习算法(高级)	✓	2s	恭喜, 完成了迷宫

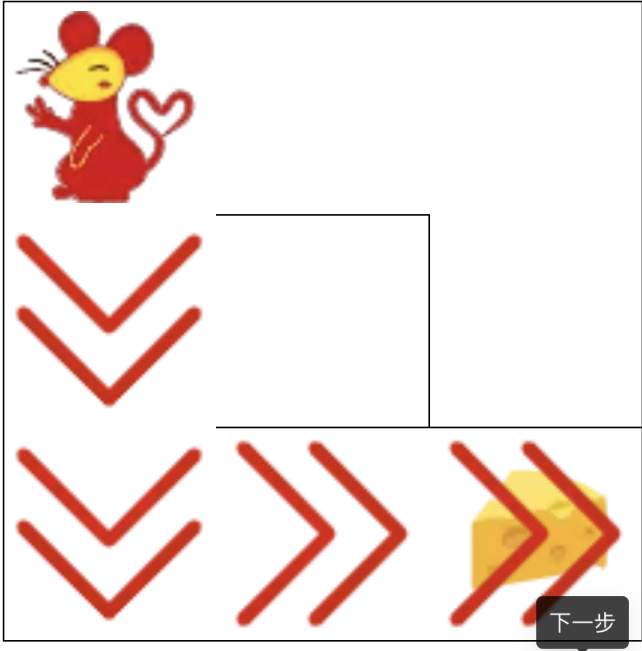
确定

这里展示一下迷宫：

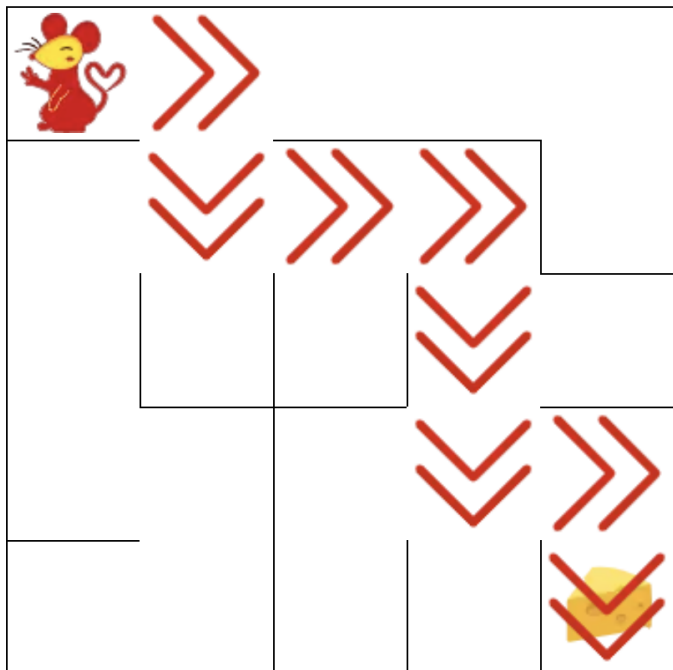
基础搜索算法 (Victory)



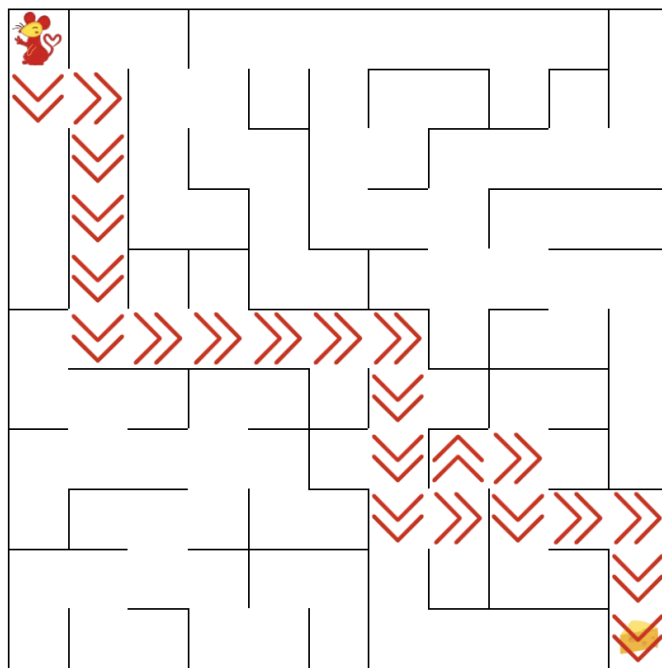
强化学习level3 (Victory)



强化学习level5 (Victory)



强化学习level11 (Victory)



五、总结

- 本实验做了超级超级久，这老鼠也太笨了！
- 实验中主要是遇到两个问题：一直撞墙和来回踱步。但是这两个问题根本还是训练不够，或者奖励函数设置不对。
- 但是本次作业很难受的就是它在平台限制了我的训练轮次以及每轮能走的步数。网上的方法大多都采用while的方式，但是这样会让学习时间大大增加，这样就失去算法的意义了，不如直接搜索算法。
- 为了解决训练不够的问题，我在初始化的时候先进行预训练，其实就是自己增加了训练轮次。但是同时用分段的思路，小地图训练少，大地图训练大一点，同时保证时间的合理程度。
- 最后一个实验了，最后真心感谢308课室的助教老师一学期的帮助和回答。306的助教也辛苦改一学期作业了。